Implementación de Pulso con GitHub Spec Kit

Caso práctico grupal de desarrollo guiado por especificaciones
29 de octubre de 2026

Qué desarrollamos y cómo comenzamos

Para esta actividad desarrollamos Pulso, una página que permite consultar criptomonedas y seguir sus cambios de precio. Utilizamos CoinGecko para obtener el precio en dólares, la variación de las últimas 24 horas y el volumen. A estas consultas les añadimos gráficos de 24 horas, 7, 30 y 90 días, además de una lista de favoritos. Nuestro objetivo fue aplicar el flujo SDD en una aplicación que pudiéramos ejecutar y comprobar con datos reales.

Comenzamos por el repositorio oficial de GitHub Spec Kit e instalamos su herramienta Specify CLI. Inicializamos el proyecto con la integración de Codex y los scripts de PowerShell. Esto nos dejó la estructura de trabajo, las plantillas y las instrucciones para desarrollar a partir de especificaciones. La versión utilizada fue la 1.2.5.

Con esa base describimos la aplicación que queríamos construir y generamos los documentos del proyecto con ayuda del asistente. Entre ellos quedaron la especificación de requisitos, el plan técnico, el modelo de datos, el contrato con la API y la lista de tareas. La constitución reunió las reglas generales, como utilizar datos verificables, identificar los errores y evitar que una captura guardada se presentara como una cotización nueva.

Estos archivos nos sirvieron para concretar el encargo. Por ejemplo, el enunciado mencionaba criptomonedas y acciones, pero nosotros trabajamos con las consultas de criptomonedas de CoinGecko. Por eso definimos BTC y ETH como ejemplos principales y dejamos las acciones bursátiles fuera de esta versión. Cuando se busca AAPL, la página explica esa limitación en lugar de mostrar otro activo con un nombre parecido.

También definimos qué debía ocurrir cuando faltaran datos. La página no debía reemplazar un valor desconocido por cero ni inventar una serie para completar el gráfico. Esta decisión terminó siendo útil al probar la aplicación, porque el proveedor podía limitar las peticiones y no siempre era posible obtener una respuesta nueva.

En la especificación organizamos el trabajo en cuatro partes: consulta del mercado, análisis de históricos, seguimiento mediante favoritos y documentación. A partir de ellas quedaron 13 requisitos funcionales y cinco condiciones de éxito. Así tuvimos una referencia concreta para revisar la aplicación y no depender solamente de que su apariencia nos pareciera correcta.

Del plan a la aplicación en funcionamiento

Una vez definida la aplicación, continuamos con el plan y las tareas de implementación. Elegimos HTML, CSS y JavaScript porque eran suficientes para el alcance del proyecto. No necesitábamos cuentas de usuario ni una base de datos: las cotizaciones se consultan en CoinGecko y los favoritos se conservan en el navegador. Para ejecutar la página localmente utilizamos un servidor sencillo de Node.js.

La estructura separó la lógica de datos de la interacción con la página. En core.js quedaron las consultas, el formato de los valores y los cálculos. En app.js se manejan la búsqueda, el activo seleccionado, la tabla y los gráficos. Esta separación nos facilitó comprobar los cálculos sin tener que revisar toda la interfaz cada vez.

Con la descripción y los documentos como referencia, utilizamos el asistente para construir el código. Después ejecutamos el proyecto y revisamos su comportamiento. La generación no resolvió por sí sola toda la entrega: hubo que comprobar los datos, ajustar la presentación y corregir problemas encontrados durante las pruebas. Los documentos también se completaron durante el desarrollo; no fue una secuencia automática en la que cada fase quedó cerrada sin volver a revisarla.

Primero quedó disponible el listado del mercado con el precio, la variación y el volumen. Luego incorporamos la selección de una moneda y su gráfico. Para cada temporalidad se consulta un número distinto de días, de modo que cambiar de 7 a 30 días modifica el histórico solicitado. El gráfico permite ver precio o volumen y consultar la fecha y el valor de una muestra.

Al trabajar con los porcentajes distinguimos dos datos. La variación de 24 horas viene de la consulta de mercado; el rendimiento del período se calcula con la primera y la última muestra del rango elegido. Si estamos viendo 30 días, ese rendimiento corresponde a esos 30 días. El volumen, por su parte, representa una ventana móvil de 24 horas y no una cantidad independiente para sumar en cada punto.

Añadimos caché para evitar consultas repetidas y mensajes para los casos de error. También obtuvimos una captura real de 50 activos y de los cuatro históricos de Bitcoin. Si CoinGecko no permite una consulta nueva, la página puede usar ese respaldo indicando su fecha. Después de las pruebas publicamos la aplicación en Sites y conservamos los registros de ejecución junto con el resto de la documentación.

Revisión de requisitos diseño y código

Para organizar la revisión tomamos como referencia el artículo de Piskala sobre SDD. Allí se describen las fases de especificar, planificar, implementar y validar, con revisión entre ellas. La parte dedicada a Kiro plantea preparar requisitos, diseño y tareas antes de generar código. Aunque nuestra herramienta fue Spec Kit, esa organización nos sirvió para revisar las decisiones del proyecto.

En los requisitos, el primer punto fue comprobar que lo escrito coincidiera con la aplicación solicitada. Revisamos la búsqueda por nombre o símbolo, la moneda USD, los datos de 24 horas y los cuatro rangos del gráfico. También consideramos qué debía ver el usuario si buscaba algo inexistente o si la API fallaba. Esto ayudó a que los errores formaran parte del comportamiento previsto y no fueran detalles que se atendieran únicamente al final.

En el diseño analizamos las consecuencias de las decisiones técnicas. Guardar favoritos en el navegador simplifica la aplicación, pero significa que no se comparten entre dispositivos. Consultar CoinGecko directamente permite una entrega sencilla, aunque dependemos de sus restricciones de acceso. Si el proyecto necesitara una clave privada, tendríamos que incorporarla mediante un servidor, porque no sería adecuado dejarla expuesta en el código del navegador.

En el código generado nos concentramos en comprobar cómo se resolvían esas reglas. Un caso fue el cálculo del rendimiento: con precios de 100, 80 y 120, el resultado esperado es un mínimo de 80, un máximo de 120 y una subida cercana al 20 %. La función se comprobó con ese ejemplo y con situaciones sin muestras o con un precio inicial igual a cero. Así pudimos revisar la lógica con resultados que se entienden sin depender del dibujo del gráfico.

También revisamos las respuestas tardías. Si cambiamos de moneda o de rango mientras una consulta sigue en curso, una respuesta anterior podría llegar después y reemplazar los datos actuales. El código utiliza una secuencia para identificar las peticiones y descartar las que ya no corresponden a la selección. Comprobamos el mecanismo y el cambio rápido de rangos, aunque todavía sería conveniente automatizar ese caso en la interfaz.

La discusión del código mostró que generar una solución no equivale a terminar de revisarla. La separación de las consultas y los cálculos resultó útil para las pruebas, mientras que app.js reúne bastante trabajo de interfaz y podría dividirse si la aplicación crece. Para esta entrega mantuvimos esa estructura porque permite entender el recorrido de los datos sin añadir más componentes de los necesarios.

Pruebas realizadas y resultado del trabajo

Para validar el proyecto ejecutamos la comprobación de sintaxis y el conjunto de 20 pruebas automatizadas. Todas aprobaron. Estas pruebas cubren los rangos permitidos, la limpieza de muestras, los cálculos, la búsqueda, los favoritos, la caché y distintos errores de las consultas. Para los fallos HTTP usamos respuestas simuladas, de manera que fuera posible repetir el caso sin esperar a que el proveedor tuviera un problema real.

En la página comprobamos la búsqueda de BTC y ETH, la selección de Ethereum, el cambio entre temporalidades y la vista de volumen. También verificamos que los favoritos permanecieran después de recargar y que el gráfico pudiera recorrerse con el teclado. La búsqueda de AAPL mostró la explicación del alcance. Durante el uso intensivo apareció una restricción real de CoinGecko y pudimos observar el funcionamiento de la captura guardada.

Las pruebas permitieron corregir problemas visibles. En móvil detectamos que la tabla provocaba desplazamiento horizontal: el contenido ocupaba 492 píxeles en un espacio de 375. Ajustamos el contenedor y el ancho de los nombres; después, el ancho disponible y el contenido quedaron en 375 píxeles. También se corrigió la recuperación de favoritos que no aparecían dentro del primer listado de monedas.

Además encontramos una limitación en el formato de precios muy pequeños. Como se muestran hasta seis decimales, un valor de 0,00000001 USD puede aparecer como $0.0000. Este caso no estaba incluido en las 20 pruebas y quedó identificado para una siguiente corrección. La solución sería adaptar la cantidad de decimales y añadir una prueba que evite mostrar un valor positivo como si fuera cero.

Al finalizar contamos con la página, el código y los documentos que explican su construcción. La aplicación permite consultar el mercado y explorar los gráficos, pero sigue dependiendo de la disponibilidad de CoinGecko. Los registros de pruebas y las capturas muestran qué se comprobó y ayudan a distinguir esos resultados de los aspectos que todavía pueden mejorarse.

Lo más útil de trabajar con SDD fue tener una referencia para justificar y revisar lo que íbamos construyendo. Spec Kit nos ayudó a organizar el proyecto y a pasar de la descripción de la aplicación a documentos y tareas. La ejecución y las pruebas nos permitieron contrastar lo generado con lo que necesitábamos. El trabajo dejó claro que esa revisión sigue siendo necesaria incluso cuando el código funciona y las pruebas automatizadas aprueban.

Referencias

Piskala, D. B. (2026). Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants. arXiv:2602.00180v1, pp. 3-5. https://doi.org/10.48550/arXiv.2602.00180

GitHub Spec Kit: https://github.com/github/spec-kit. CoinGecko API: https://www.coingecko.com/en/api.

Documentación del proyecto: specs/001-crypto-tracker/, tests/core.test.mjs y docs/evidence/. Aplicación: https://pulso-mercado-sdd.deepgiant66.chatgpt.site.